SeGen: Automatic Topology Generator for Sequencing Elements

Kyounghun Kang
Korea Advanced Institute of Science and Technology
School of Electrical Engineering
Daejeon, South Korea
rudgns546@kaist.ac.kr

Wanyeong Jung
Korea Advanced Institute of Science and Technology
School of Electrical Engineering
Daejeon, South Korea
wanyeong@kaist.ac.kr

ABSTRACT

Sequencing elements, such as flip-flops (FFs), significantly impact the speed, size, and power consumption of digital integrated circuits. Despite numerous sequencing-element proposals in the past, they have heavily relied on the expertise of designers, and their design method could not search all available FF topologies. This paper introduces a design framework for automatically generating sequencing element designs, named SeGen. Motivated by the fact that the operation of all digital circuits can be represented by a Boolean function, we present SeGen which initially generates all possible Boolean functions for sequencing elements and derives circuit topologies from each of them. By adjusting the functionality qualification step of Boolean functions, SeGen can generate other sequencing elements such as toggle FFs and dual-edge-triggered FFs as well. A total of 40 resulting topologies newly generated by SeGen encompass the entire spectrum of positive-edge-triggered FFs, including master–slave FFs and pulsed latches, and consequentially provide a range of designs suitable for diverse applications. Several generated FFs such as SeGen8 and SeGen28 outperform recent human-crafted FFs and exhibit energy–delay product (EDP) improvement of 203%–257% at 1V supply voltage, compared to conventional transmission-gate FF (TGFF).

KEYWORDS

Flip-flop, pulsed latch, finite-state machine, automatic topology generation, energy efficiency.

1 INTRODUCTION

As device scaling approaches its limitations [3, 19], there is a growing desire to develop new circuit topologies for achieving meaningful performance improvements. Automatic topology synthesis of circuit blocks is a promising solution with this trend [4, 5, 8, 13, 14], enabling fast and efficient exploration of a broader range of circuit topologies. A designer can easily choose the most suitable circuit topology among the outcomes based on the application specifications.

Among important circuit blocks, a flip-flop (FF) is a compelling option for design automation owing to its popularity and potential benefits regarding the speed, power consumption, and size of the overall digital system [1, 18]. The transmission-gate FF (TGFF), which features a master–slave edge-triggered structure, stands as the prevailing choice for sequencing cells because of its straightforward operation and robustness. Over the decades, many sequencing elements have been developed with specific design goals, such as low-power or high-speed, but unexpected side effects persist.

As an illustrative example of dynamic power reduction, the true single-phase clock (TSPC) FF [22] uses dynamic logic with a single-phase clock, allowing positive-edge-triggered actions without the need for two-phase clocks. However, this configuration results in reliability issues due to floating nodes inside the circuit. In a subsequent study, a single-phase static contention-free FF (S2CFF) [11] enhances the stability of the dynamic FF, whereas it is ineffective for resolving redundant transitions inside the FF. Another design, change-sensing FF (CSFF) [12], focuses on eliminating redundant internal transitions to achieve power reduction. However, an unforeseen reliability issue (due to its dynamic operation) arises in exchange. A redundancy eliminated FF (REFF) [17] acknowledges these patterns and succeeds in optimization to some extent (reliability and low-power), but eventually, its extended setup time as a trade-off limits its application in high-speed scenarios.

As evident from the preceding examples, there is no ideal FF design, and the optimal FF design can vary depending on the requirements, such as low-power or high-speed. Conversely, if we could generate as many FF topologies as possible to cover a wide range of design specifications and select the more suitable design for the application rather than crafting a specific-purpose FF, both the circuit's performance and the design cost could be improved.

As part of an earlier study to create new FF topologies, efforts have been directed toward generating a new FF topology from a specific gate-level representation and its compound-gate-level implementation [2, 10]. In contrast to transistor-level design, which adjusts individual transistors in a predefined FF topology [11, 12, 17] (Figure 1, top left), the gate-level design enables developing a whole topology from its behavior described in a gate-level representation (Figure 1, bottom left). To minimize the transistor count of the compound-gate-level implementation, a topologically-compressed FF (TCFF) [10] merges transistors with equivalent functions at the transistor level (referred to as the topological compression method), while an 18 true single-phase clock FF (18TSPC) [2] employs an equivalent node merging technique at the gate-level. However, both structures are designed on the basis of a pre-determined gate-level representation indicating a master–slave edge-triggered operation; thus, it still proves challenging to design other types of sequencing elements, such as a pulsed latch (e.g., self-timed pulsed latch (STPL) [7] and differential contention-free pulsed latch (DCPL) [15]), in the synthesis process outlined in the aforementioned papers.

In this study, we introduce SeGen—an automatic topology generator for sequencing elements. The operating principle of SeGen

ICCAD '24, October 27–31, 2024, New York, NY, USA

ACM ISBN 979-8-4007-1077-3/24/10.
https://doi.org/10.1145/3676536.3676665

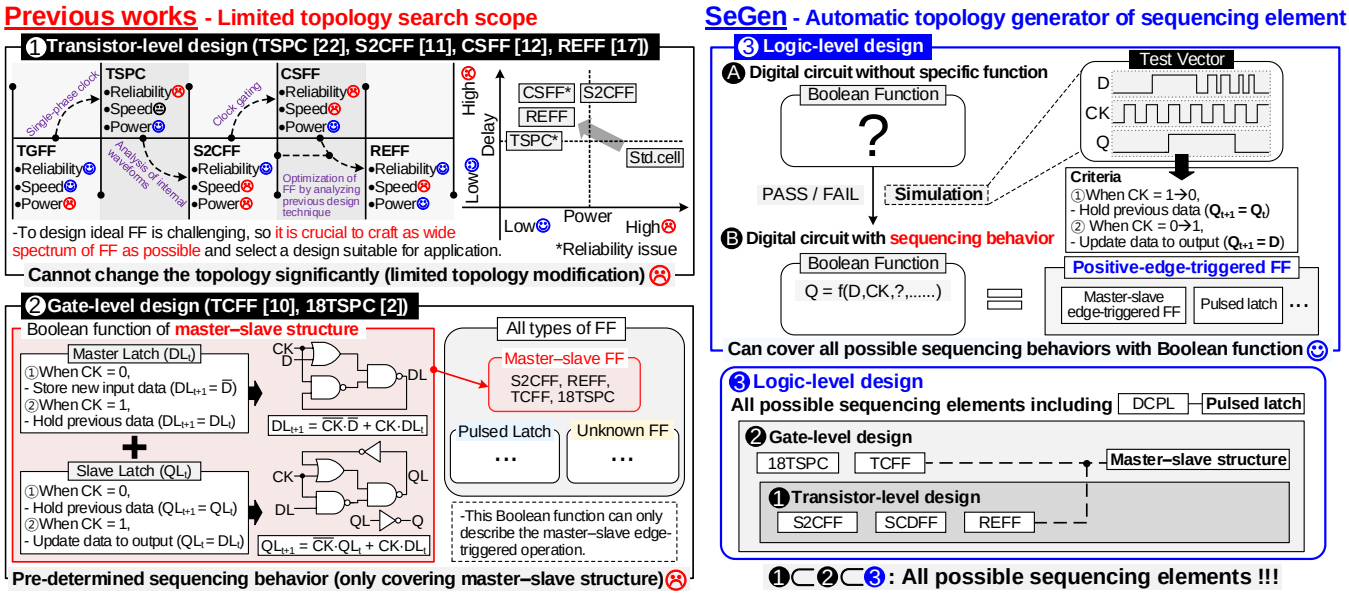


Figure 1: Overview of sequencing element synthesis: SeGen produces all possible Boolean functions by transforming the transistor/gate-level description into higher-level description.

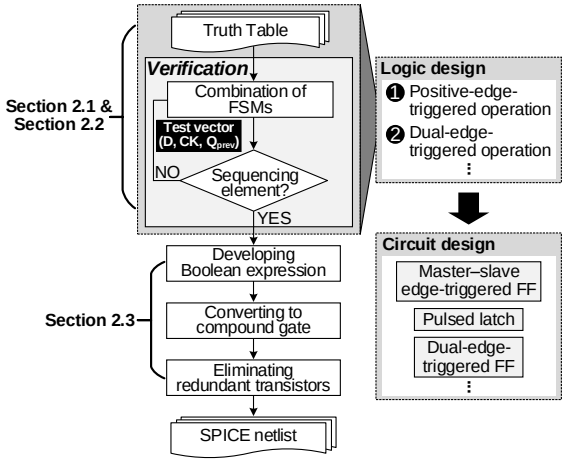


Figure 2: Flowchart of sequencing element synthesis.

is based on a key observation that the fundamental mechanism of a flip-flop circuit can be effectively represented as a Boolean function (Figure 1, right); thus, SeGen produces all possible Boolean functions (sequencing mechanisms) of sequencing elements by solely observing its behavior (output, Q). The Boolean functions generated by SeGen are then translated into transistor-level schematics. Through this process (Figure 2), we have discovered all other structures capable of sequencing behavior, including the traditional master–slave edge-triggered operation and pulse-triggered operation. Through transistor-level simulations, the outcomes generated by SeGen can be judiciously selected for specific applications, considering factors such as power, speed, and other design constraints. The key contributions of this paper are outlined as follows.

- An automated FF behavior search enables finding all possible FFs without analyzing their internal behaviors.
- The SPICE simulation results verify that SeGen can generate FFs with specifications that existing structures cannot cover.
- The resulting topologies from SeGen guarantee reliable operation without the need to check each transistor's operation, as it has been validated at a higher-level description.
- This framework also can be applied to other types of sequencing elements, such as dual-edge FFs, by adjusting the validation criteria of a Boolean function.

The remainder of this paper is arranged as follows. In Section 2, we describe the process of synthesizing FFs using Boolean functions extracted from SeGen. Section 3 shows the experimental results and discussions, and Section 4 concludes the paper.

2 SEQUENCING ELEMENT SYNTHESIS METHODOLOGY

Recognizing the limitations in topology synthesis from previous research, SeGen explores all design spaces of the sequencing elements by starting the design at the logic-level (level of fundamental behavior). The comprehensive sequencing element synthesis process is depicted in Figure 2. Any logic behavior can be described as a truth table; thus, SeGen commences with all possible truth tables and assesses whether each of them can effectively represent the desired sequencing behavior. To validate their functionality, a carefully designed test vector of input data (D) and clock (CK) is employed to cover all transitions of inputs and outputs. Following this, each qualified truth table is translated into a gate-level representation and a compound gate implementation. Transistor-level optimization is performed afterward by combining duplicated transistors and internal nodes based on their electrical behaviors in response to the inputs and internal states. The resulting transistor-level topologies are evaluated through SPICE simulation and compared with the TGFF and other state-of-the-art FFs. Details of each process are provided in the following subsections.

2.1 Preliminaries

A widely used and reliable way to describe the behavior of a digital system is a Boolean function using "0", "1", and their logical

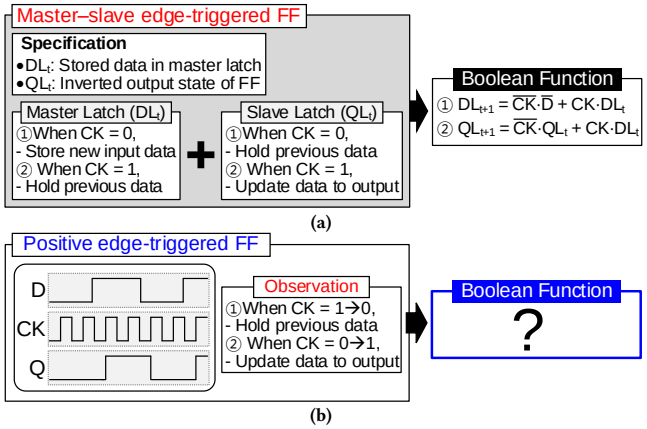


Figure 3: (a) Gate-level representation describing the master–slave edge-triggered FF's operation, (b) SeGen searches all possible Boolean functions to describe the sequencing behavior with the only observation of the output (Q).

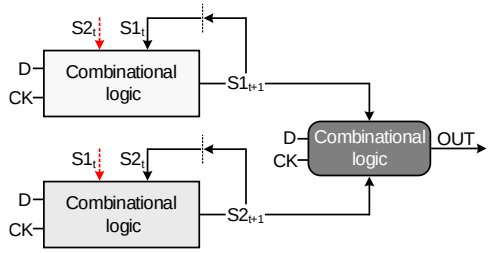


Figure 4: Initial setting of sequencing element synthesis with two-states.

combinations ($B = \{0,1\}$, f: $B^n \rightarrow$ B). For example, the fundamental behavior of a master–slave edge-triggered FF is to store new input data (D) in the master latch when the clock (CK) is low and transmit it to the slave latch when CK is high (Figure 3a). This operation can be described as Boolean functions (truth tables) of D, CK, the output (Q), and some internal states, which can be then translated into gate- or transistor-level representations.

This implies, in other words, any new FFs based on the Boolean function of the master–slave FF cannot be a different type of sequencing element such as a pulsed latch. On the other hand, this study searches all feasible Boolean functions solely according to the output of the sequencing element without predicting its internal operation (as shown in Figure 3b). This significantly expands the coverage of the proposed framework and allows searching for any type of sequencing element by adjusting the qualifying test vector. In this study, we focus on positive-edge-triggered FFs as they are widely used.

To establish initial truth tables, the number of internal states needs to be determined. Several experiments have been performed to establish this initial configuration. When exclusively composed of combinational logic (no internal state), sequencing behavior is not possible since the changes in the input data (D) and the clock (CK) are instantaneously reflected in the output. When there is only a single state, no truth table passes the qualification, and it turns out to have insufficient information for the edge-triggered operation.

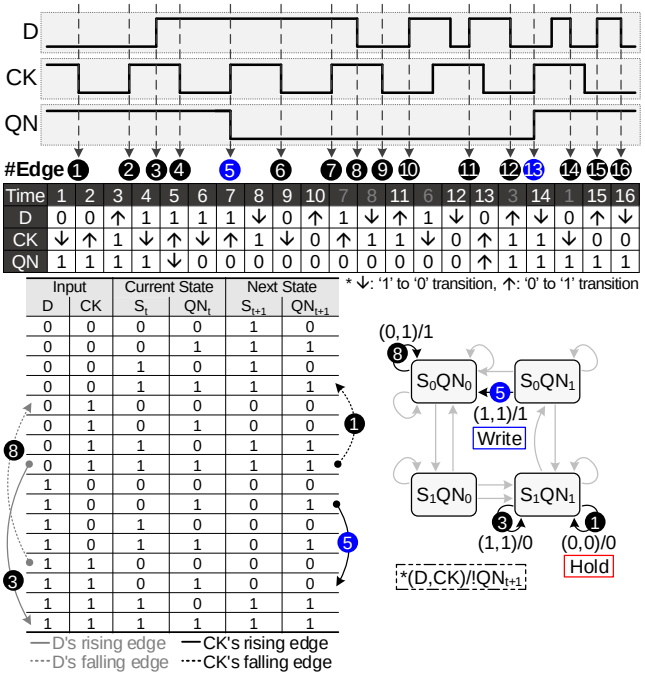


Figure 5: All possible scenarios of the sequencing element and state transition at the edges of D and CK and the example of edge tracking.

In the next step, the test setup is configured as shown in Figure 4 so that a presumptive FF has two internal states (each without any special assumption regarding its behavior) and an output from the two states and inputs.

Our experiment has found that this configuration can construct truth tables qualified as those of FFs, and every truth table has a state that consistently mirrors the output of the sequencing element (Q) or its complement (QN). Thus, in the following discussion, one state is always presumed to be the inverted output (QN) with minimal loss of generality (because the output of the sequencing element should be extracted through an inverter acting as a buffer), and the other random state is denoted as S. This assumption and notation allow an easier description of the remaining steps and results.

2.2 Validation of Sequencing Element

Without the need to investigate the internal behavior, a truth table is validated by simply observing its output with a sequence of input combinations of D and CK. As illustrated at the top of Figure 5, there are 16 possible transition scenarios for a FF, and the test vector is designed to cover all of them.

The validation algorithm commences by searching for stable states. In Figure 5, the initial input is D = 0, CK = 1 (before Edge 1). Because $(S_t,QN_t) = (S_{t+1},QN_{t+1}) = (1,1)$ with these inputs (Row 8 of the truth table in Figure 5), it can be established as an initial state. Similarly, (S,QN) = (0,0) can be another initial state (Row 5), which will be discussed later.

When the first input edge comes in (Edge 1), CK falls (1→0) and the next states (S_{t+1},QN_{t+1}) are generated according to the corresponding row in the truth table (Row 8→Row 4). The resulting states remain unchanged at (S,QN) = (1,1). There is no change in QN, indicating that it successfully passes the Edge 1 test (Q must be

Algorithm 1: Validation of FF

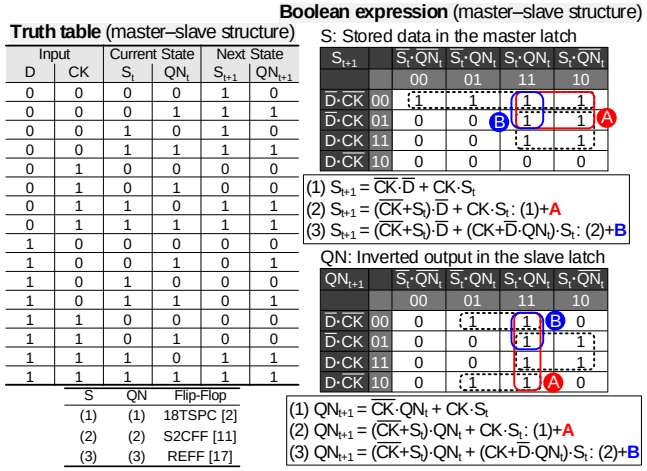
Input: Truth table = $[t_1, t_2, \ldots t_N]$;
Output: Truth table of FF *Seq*;
for $1 \le i \le N$ **do**
  for *state* ***in*** *stable_initial_states* **do**
    Initialize state S_tQN_t;
    edges = $[e_1, e_2, \ldots e_{16}]$;
    for *edge* ***in*** *edges* **do**
      while *Have not reached the stable state* **do**
        Update state S_tQN_t by tracking edges;
        Save the history of state transition;
        if *Revisit past state* **then**
          Activate failure signal;
          Break the inner closed loop;
      if *Have not reached the stable state* **then**
        Activate failure signal;
        Break the inner closed loop;
      if *State QN arrives at the unwanted state* **then**
        Activate failure signal;
        Break the inner closed loop;
    if *Failure signal is not activated* **then**
      Seq.append (t_i);
      Break the inner closed loop;
return *Seq*;

changed only by positive edges of CK). The next edges subsequently trigger state updates, and the change of QN is tested if it follows that of a positive-edge-triggered FF. For example, in the case of an output transition (as seen in Edge 5), the starting state is (S,QN) = (0,1) as a result of Edge 4 (Row 10). When CK rises, the state transitions are as follows: (S,QN) = (0,1)→(0,0) (Row 14)→(0,0) (Row 13). There is a change in the output (Q) after the rising edge of CK, implying the passage of the Edge 5 test. To determine truth tables of valid FFs, these processes are repeated and evaluated across all 16 edges of the test vector. An unexplored row of truth table in the testing procedure is regarded as an unreachable state because it does not affect the operation of FFs. If a truth table does not pass the FF test with an initial state, the test is repeated with another possible initial state until it passes. Algorithm 1 details the validation process.

2.3 Converting Logic Expression into Transistor-Level Design

After all Boolean functions (truth tables) describing the sequencing behavior have been identified, they are converted to corresponding gate-level representations. A truth table can be transformed into multiple gate-level representations, resulting in transistor-level topologies with different performance metrics. This section describes how to construct gate-level representations and further optimize transistor-level FF circuits.

2.3.1 Gate-level Representation of Sequencing Element. Because the operations of an FF generated by SeGen are described as Boolean logic, the internal states are always connected to VDD or GND when implemented as a compound gate from gate-level representation. Thus, the resulting topologies from SeGen are free from reliability issues caused by floating nodes, which are observed in previous



Input		Current State		Next State	
D	CK	S_t	QN_t	S_{t+1}	QN_{t+1}
0	0	0	0	1	0
0	0	0	1	1	1
0	0	1	0	1	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	0	1
1	0	1	0	0	0
1	0	1	1	0	1
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	1	1
1	1	1	1	1	1

S_{t+1}		$S_t\cdot\overline{QN_t}$	$\overline{S_t}\cdot QN_t$	$S_t\cdot QN_t$	$\overline{S_t}\cdot\overline{QN_t}$
		00	01	11	10
$\overline{D}\cdot CK$	00	1	1	1	1
$\overline{D}\cdot\overline{CK}$	01	0	0	1	1
$D\cdot CK$	11	0	0	1	1
$D\cdot\overline{CK}$	10	0	0	0	0

QN_{t+1}		$S_t\cdot\overline{QN_t}$	$\overline{S_t}\cdot QN_t$	$S_t\cdot QN_t$	$\overline{S_t}\cdot\overline{QN_t}$
		00	01	11	10
$\overline{D}\cdot CK$	00	0	1	1	0
$\overline{D}\cdot\overline{CK}$	01	0	0	1	1
$D\cdot CK$	11	0	0	1	1
$D\cdot\overline{CK}$	10	0	1	1	0

S	QN	Flip-Flop
(1)	(1)	18TSPC [2]
(2)	(2)	S2CFF [11]
(3)	(3)	REFF [17]

Figure 6: Various Boolean expressions of the master–slave edge-triggered FF.

FFs such as TSPC [22] and CSFF [12]. Furthermore, by modifying the gate-level representation, potential race conditions (glitches) can be completely resolved.

Figure 6 shows the truth table describing a master–slave FF and its various Boolean expressions. In a master–slave structure, S indicates the stored data in the master latch, and QN indicates the inverted output in the slave latch. A Boolean function of the master–slave edge-triggered FF appears in several forms of a Boolean expression, each creating a different topology. For example, the truth table in Figure 6, can yield several topologies that function as 18TSPC [2], S2CFF [11], and REFF [17] when each FF's gate-level implementation is based on the combination of Boolean expressions S(1)&QN(1), S(2)&QN(2), and S(3)&QN(3), respectively.

The choice of Boolean expression for a Boolean function can lead to the creation of distinct topologies with a peculiar feature. For example, 18TSPC features a simplified structure owing to the minimum sum-of-products (SOP) gate representation, but the race condition can occur when D = 0 and CK = 0→1 ([17]). By adding $(/D\cdot S_t)$ and $(QN_t\cdot S_t)$ terms in S(1) and QN(1), respectively (these terms are bridge terms to group nonoverlapped product terms), hazard-free Boolean expressions are finally completed. The resulting flip-flop (e.g., S2CFF [11], S(2)&QN(2)) therefore ensures reliable operation without floating node or glitches. REFF additionally includes a conditional term $(/D\cdot QN_t\cdot S_t)$ in each of S(2) and QN(2) Boolean expressions. This conditional term is eventually converted into a circuit with a specific function to eliminate redundant transitions when D = Q. As shown in the examples above, How the Boolean expression is articulated holds the essence of an FF design. This is because it determines the core feedback structure of a sequencing element topology.

2.3.2 Compound Gate. Typically, compound gates such as AOI or OAI are used to implement a compact FF circuit. Before implementing the final FF compound gate, it is necessary to discuss the common terms included in each Boolean expression such as S&QN. Since these common terms eventually form internal nodes with the same role, they are regarded as equivalent nodes and merged, thereby implementing a transistor-level schematic. (Steps 1&2 in Figure 7).

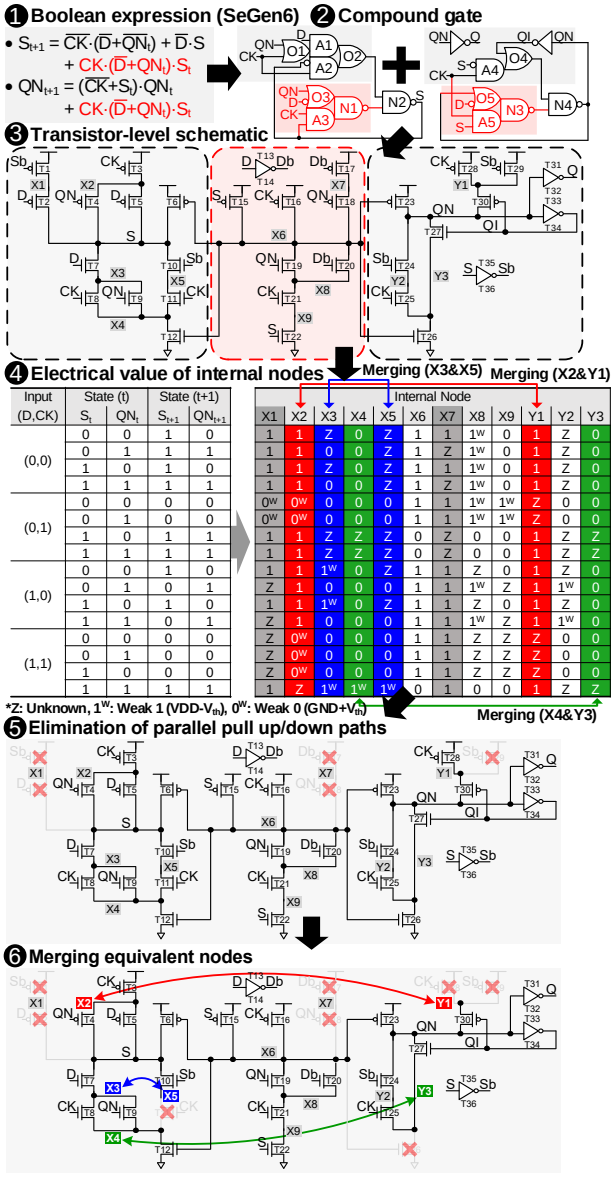


Figure 7: Example of the logic synthesis procedure for a positive edge-triggered-FF (e.g., SeGen6).

2.3.3 Elimination of Redundant Transistors. After a compound gate schematic is developed out of a truth table, redundant transistors in the transistor-level schematic are eliminated. The fundamental principle of eliminating redundant transistors is to combine overlapped transistors or equivalent nodes with the same role. This is achieved by checking the electrical values of all internal nodes in the transistor-level schematic based on the inputs (D, CK) and internal states (Step 4 in Figure 7). According to the principle outlined above, the transistor merging process involves two steps: A) merging multiple pull-up or pull-down paths into a single path if they are formed in parallel (**path merging**); B) combining internal nodes with the same state into a single node (**node merging**). Following is an example of the actual merging process for SeGen6 (Steps 5&6 of Figure 7).

- **Path merging1:** The pull-up paths for S node consist of PMOSs with a series of D&Sb inputs, PMOS with X6 input, and PMOSs with a combination of CK·(D+QN) inputs. Two PMOSs with Sb&D inputs (T1&T2) can be removed because the S node remains high statically owing to alternative pull-up paths (CK(=0)·QN(=0) or CK(=0)·D(=0) or X6(=0)).
- **Path merging2:** In the case of the X6 node, PMOS with S input, PMOS with CK input, and PMOSs with a series of Db·QN inputs link X6 to VDD. Two PMOSs with a series of Db&QN inputs (T17&T18) can be eliminated because other pull-up paths (S(=0) or CK(=0)) are always activated when X6 = 1.
- **Path merging3:** Finally, the PMOS with Sb input (T29) can be disregarded because another pull-up path for QN always exists when QN_t = 1. (When CK = 0, PMOSs with a series of CK&QI inputs (T28&T30) operate. When CK = 1, the PMOS with X6 input (T23) operates).

To further eliminate redundant transistors, the internal nodes with identical states can be combined. An unknown value (Z) implies that there is no conduction path connected to other fully static nodes (S or QN) or VDD or GND; thus, it can be regarded as a "Don't Care" value, which can be 0, 0^{Weak}, 1^{Weak}, or 1 (Step 4 in Figure 7).

- **Node merging1:** When X2 and Y1 share nodes, parallel pull-up paths to VDD exist, and one of them (T28) can be eliminated.
- **Node merging2:** When combining X3 and X5 nodes, parallel pull-down paths exist in S, which results in the removal of the NMOS with CK input (T11).
- **Node merging3:** When X4 and Y3 share nodes, parallel pull-down paths to GND exist, and one of them (T26) can be eliminated.

When the redundancy elimination process is complete, the final FF circuit topology is obtained.

3 EXPERIMENTAL RESULTS

3.1 Various Topologies from SeGen

SeGen has identified a total of 114 Boolean truth tables with two internal states (S and QN) that show sequencing behaviors. When the test vector in Figure 5 is applied, truth tables with identical internal state transition behavior are grouped, which eventually converge to 40 representative Boolean functions (Table 1). From each Boolean function group, the transistor-level circuit is implemented in accordance with the sequencing element synthesis procedure (Figure 2). To prove the versatility of the set of complete truth tables and power of the automated SeGen framework, transistor-level schematics have been generated based on their minimum SOP gate representations (plus additional terms for glitch-free operation).

As discussed in Section 2.3.1, a broader range of FFs can be discovered by further exploring different gate-level Boolean expressions by a designer or an automated tool, including recently published FFs [2, 10, 11, 15, 17] and various other designs that have not been discovered thus far. The resulting topologies are categorized into the following three types.

a) Flip-flops based on a StrongARM latch. Their operating principle (truth table) is the same as that of a master–slave FF. They receive differential input data, and the rising edge of CK triggers output changes. This type of FF is usually generated when a gate

Table 1: 40 Boolean truth tables from SeGen (2×20).

Input		State (t)		State (t+1)																															
				SeGen1		SeGen2		SeGen3		SeGen4		SeGen5		SeGen6		SeGen7		SeGen8		SeGen9		SeGen10		SeGen11		SeGen12		SeGen13		SeGen14		SeGen15		SeGen16		SeGen17		SeGen18		SeGen19		SeGen20	
D	CK	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN
0	0	0	0	1	0	1	0	1	0	1	0	0	0	1	0	1	0	1	0	1	0	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	0	0	0
0	0	0	1	1	1	0	1	0	1	1	1	0	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1	1	0	1	0	1	1	1	0	1	0	1
0	0	1	0	1	0	1	0	1	0	1	0	0	0	1	0	1	0	1	0	1	0	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	0	0	0
0	0	1	1	1	1	0	1	0	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	0	1	1	1	1	1	0	1	1	1	1	1	0	1	1	1	0	1
0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	1	0	1	0	0	1	1	1	1	0	0	0	1	0	0	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1	1
1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
1	0	0	1	0	1	0	1	0	1	0	1	1	1	0	1	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
1	0	1	0	0	0	0	0	1	0	1	0	0	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
1	0	1	1	0	1	0	1	0	1	0	1	1	1	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1

Input		State (t)		State (t+1)																															
				SeGen21		SeGen22		SeGen23		SeGen24		SeGen25		SeGen26		SeGen27		SeGen28		SeGen29		SeGen30		SeGen31		SeGen32		SeGen33		SeGen34		SeGen35		SeGen36		SeGen37		SeGen38		SeGen39		SeGen40	
D	CK	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN	S	QN
0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	1	1	0	1	0	1	0	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	0	1	1	1	0	1	0	1	1	1
0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
0	1	1	0	1	1	1	1	0	1	1	1	1	0	1	1	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0
0	1	1	1	1	1	1	1	0	1	1	1	1	0	1	1	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0
1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
1	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	1	0	1	0	1	0	1	1	1	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0
1	1	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1	1	0

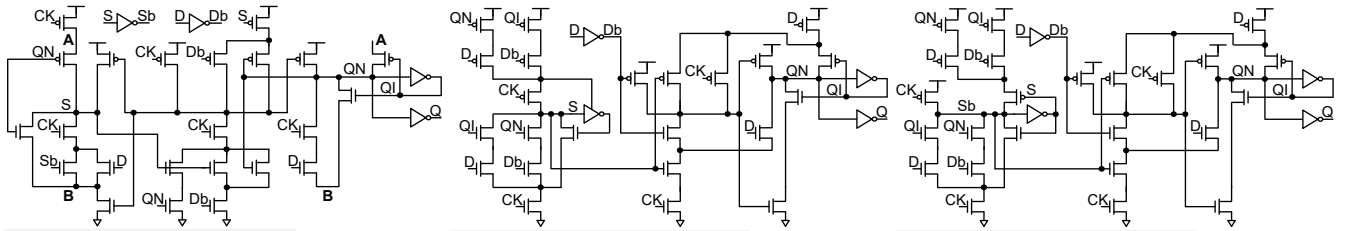


Figure 8: Toplogy examples from SeGen: (a) SeGen8, (b) SeGen11, (c) SeGen28.

	Supply Voltage (V)								
	1	···	0.8	0.75	···	0.45	0.4	0.35	0.3
SeGen(3,9,14, 15,16,20,22)	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	FAIL
Rest of SeGen	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
TGFF	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
18TSPC [2]	PASS	PASS	PASS	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
STPL [7]	PASS	PASS	PASS	PASS	PASS	PASS	PASS	FAIL	FAIL
TCFF [10]	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
S2CFF [11]	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
CSFF [12]	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
DCPL [15]	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
REFF [17]	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS

Figure 9: Reliability test of SeGen's outputs including previously reported FFs [2, 7, 10–12, 15, 17] and TGFF.

representation is expressed as S_{t+1} = /CK·D + D·S_t + CK·S_t and QN_{t+1} = /CK·QN_t + QN_t·/S_t + CK·/S_t. In the logic synthesis of truth tables, SeGen25 is constructed according to these Boolean expressions, and topologies generated from such gate representations include a static-contention-free differential FF (SCDFF) [16] among other recent publications.

b) Single-ended master–slave FFs (with suppressed redundant transitions). Their sequencing behavior is based on the master–slave structure, but some of them change their internal states less frequently, relying on a gated clock signal. The FF has a combinational logic output of (D, CK, S, QN) and can have a clock-gating conditional term (e.g., /D·QN·S) in the gate-level representations to eliminate redundant internal transitions. Most recent low-power FFs [2, 11, 12, 17] have this kind of operation mechanism, including SeGen1.

c) Pulsed latches and others. Unlike aforementioned structures, S is affected by QN to some extent. This category contains pulsed latches [7, 15] and other FFs with triggering actions resulting from interactions between D, CK, and S. Except for SeGen1 and SeGen25, all Boolean functions generated by SeGen belong to this category. All sequencing behaviors (truth tables) except for pulse-triggered operation have been newly discovered by SeGen, and several topologies (SeGen8, SeGen11, and SeGen28 in Figure 8) in this category have exhibited superior performance to existing state-of-the-art designs.

By changing the edge condition of the test vector, other types of sequencing elements can be identified too. As an example, we have explored truth tables for dual-edge FFs (with two internal states as shown in Figure 4) and identified a total of 912 Boolean functions, among which a commonly used dual-edge FF (Latch-MUX type)

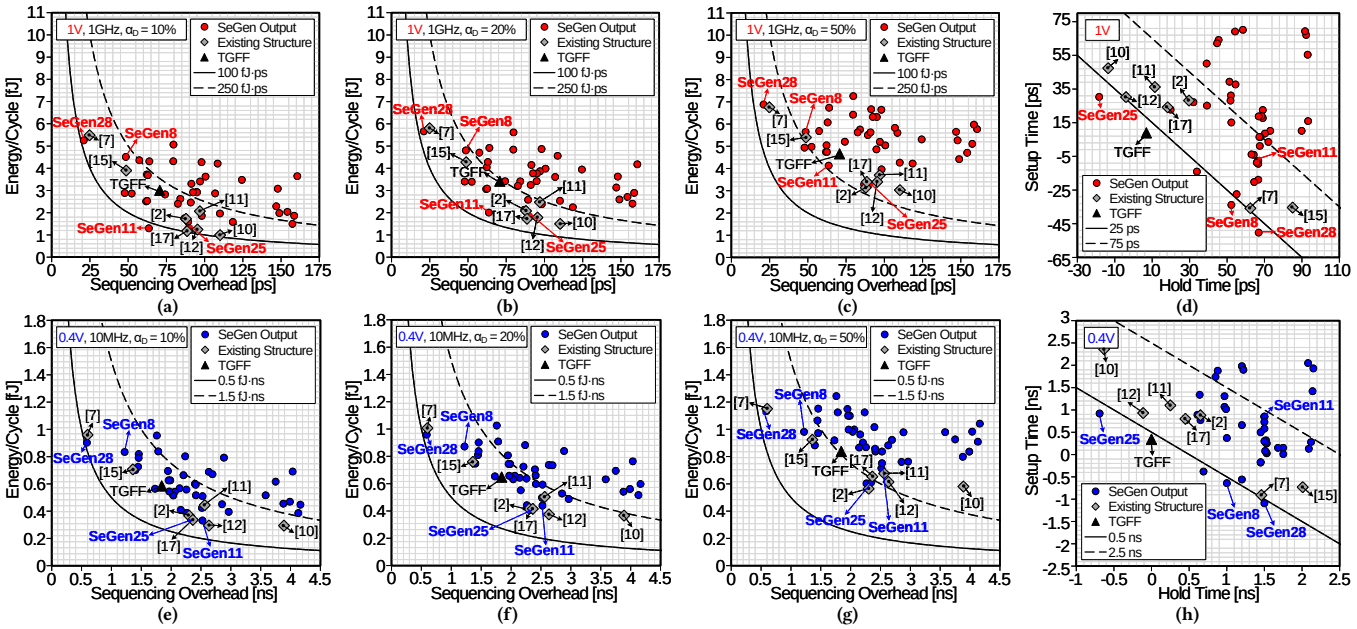


Figure 10: SPICE simulation results for SeGen's outcomes including existing structures. (a–c) Power dissipation vs. sequencing overhead at 1 V. (d) Setup time vs. hold time at 1 V. (e–g) Power dissipation vs. sequencing overhead at 0.4 V. (h) Setup time vs. hold time at 0.4 V.

exists. This demonstrates that the proposed framework (SeGen) can be utilized for other types of sequencing elements.

3.2 Simulation Setup

The resulting topologies generated by SeGen (hazard-free SOP gate representations with transistor merging), along with previously reported FF structures [2, 7, 10–12, 15, 17] and the TGFF, have been designed and compared using a 65 nm CMOS process design kit. For comparing the performance of different topologies fairly, all FFs use the standard transistor sizing (all transistors use minimum length, and PMOSs use 2× width of NMOSs). In the circuit simulations, the output loads of all the FFs have been set to FO4 (Fan-out 4).

To characterize generated FFs, their key performance metrics, including energy, setup time, hold time, clock to Q (CK-Q) delay, and sequencing overhead (setup time + CK-Q delay), have been evaluated through SPICE simulations. Energy has been calculated at various data activity ratios (α_D). The setup and hold times have been determined as the worse number between D = 0 and D = 1. The CK-Q delay is obtained in the worse case between Q's rising and falling transitions.

3.3 Comparison of SeGen's Outcomes with Existing Structures

As shown in Figure 9, 5000-run Monte Carlo simulations (global+local) have been performed to assess the reliability of resulting topologies from SeGen (test vector is same as Figure 5). Among recently published FFs, 18TSPC [2], TCFF [10], and CSFF [12] showed reliability issues, caused by their well-known internal contention and resulting functional failure at a near-threshold voltage (NTV) where the effect of PVT (process, voltage, temperature) variations are severe [6, 9, 21]. On the other hand, topologies from SeGen always guarantee static operation (it is based on Boolean logic), and all potential race conditions in the FF's operation scenarios are removed by selecting glitch-free gate-level representation. Therefore, all 40 structures generated by SeGen can reliably operate at NTV, similar to other robust FFs (TGFF, [11, 15, 17]).

Figure 10 illustrates the key performance metrics of 40 resulting topologies (from 40 Boolean functions) from SeGen and previous FFs at a supply voltage of 1 V (top) and 0.4 V (bottom). SeGen's output FFs with different sequencing behaviors are distributed over the entire range, and certain outcomes are even positioned at the range that existing structures cannot cover. This implies that a topology suitable for a specific application can be found after merely constructing topologies using SeGen.

In Figure 10a, certain SeGen's outputs (SeGen11 and SeGen28) exhibit exceptional performance (lowest energy–delay product (EDP) and lowest sequencing overhead, respectively) compared with TGFF and other state-of-the-art FFs. Unlike traditional pulsed latches that produce pulses inside the circuits on every clock cycle, SeGen11 has a Boolean function in which pulses are generated only under a certain condition (D ≠ Q). Thanks to its conditional pulse generation, SeGen11 reduces unnecessary internal transitions. Additionally, in the transistor merging step (Section 2.3.3), the unnecessary pull-down path for S formed in a situation of CK = 0 and D = Q, is eliminated. As a result, SeGen11 consumes low power while maintaining a compliant speed compared to master–slave FFs [2, 10–12, 17]. Especially, SeGen11 exhibits 126%–242% better EDP than prior arts of FFs at the 10% data activity ratio (the typical activity ratio in System-on-Chips (SoCs) [2, 10–12, 17]).

SeGen28, another pulsed latch-type topology, exhibits the best speed among all topologies. Unlike prior pulsed-latch designs [7, 15], it has a keeper at the S node resulting from the redundant transistor elimination process of transistor-level circuit. Although this keeper does not affect the functionality of the FF (it can be regarded as a

Table 2: Performance summary and comparison at 1 V.

Structure	Energy (α_D=10%) [fJ]	Energy (α_D=50%) [fJ]	Setup-Hold Window [ps]	Sequencing Overhead [ps]	EDP (α_D=10%) [fJ·ps]	EDP (α_D=50%) [fJ·ps]
TGFF	3.00	4.66	15.8	70.7	212	329
18TSPC [2]	1.72	3.14	57.8	87.8	151	276
STPL [7]	5.48	6.76	26.6	24.8	136	168
TCFF [10]	**0.99**	**3.03**	33.8	110.1	109	334
S2CFF [11]	2.06	3.70	47.6	96.9	200	359
CSFF [12]	1.24	3.40	26.2	95.4	118	324
DCPL [15]	3.89	5.40	49.6	48.7	190	263
REFF [17]	1.17	3.40	42.4	88.5	104	301
SeGen8	4.51	5.65	18.4	48.5	219	274
SeGen11	1.30	4.13	59.0	63.5	**83**	263
SeGen25	1.30	3.28	**12.0**	89.6	117	294
SeGen28	5.26	6.88	16.6	**21.0**	110	**145**

Table 3: Performance summary and comparison at 0.4 V.

Structure	Energy (α_D=10%) [fJ]	Energy (α_D=50%) [fJ]	Setup-Hold Window [ns]	Sequencing Overhead [ns]	EDP (α_D=10%) [fJ·ns]	EDP (α_D=50%) [fJ·ns]
TGFF	0.59	0.84	0.34	1.84	1.08	1.54
18TSPC [2]	0.37	**0.56**	1.53	2.30	0.84	1.30
STPL [7]	0.96	1.15	0.55	0.60	0.58	0.69
TCFF [10]	**0.29**	0.58	1.73	3.89	1.14	2.26
S2CFF [11]	0.44	0.68	1.35	2.56	1.14	1.73
CSFF [12]	0.30	0.61	0.82	2.63	0.78	1.62
DCPL [15]	0.71	0.92	1.28	1.36	0.96	1.25
REFF [17]	0.34	0.65	1.25	2.36	0.79	1.54
SeGen8	0.83	0.98	0.36	1.22	1.02	1.20
SeGen11	0.33	0.75	2.35	2.52	0.84	1.90
SeGen25	0.34	0.61	**0.23**	2.35	0.80	1.44
SeGen28	0.90	1.13	0.41	**0.59**	**0.53**	**0.67**

redundant transistor and omitted during the redundant transistor elimination step), it increases the circuit's speed (SeGen28 is 118%–524% faster than prior arts of FFs). It is also noteworthy that SeGen8 features a new sequencing mechanism different from traditional master–slave FFs and pulsed latches, and it has a good timing margin (narrow metastability window as shown in Figure 10d) as well as higher speed compared to TGFF.

When the supply voltage is scaled down to 0.4 V (Figure 10, bottom), assuming ultra low-power applications, master–slave FFs ([2, 10–12, 17], SeGen1, and SeGen25) consume less power than other topologies. Among them, SeGen25 exhibits extraordinary performance among all FFs, with the lowest hold time (key performance metric for reliable low-voltage operation), the best setup-hold window, and state-of-the-art level energy efficiency. It may lead to a significant benefit in most low-power circuits in terms of power and area. However, since these structures show significant speed degradation, they are not suitable for high-speed scenarios. On the other hand, SeGen28 still shows 104%–338% better EDP than prior arts, thanks to its fastest speed. A comparison of previous works and some good SeGen outputs is presented in Table 2 (1 V supply voltage) and Table 3 (0.4 V supply voltage) (the optimal values are highlighted in bold). SeGen(8,11,25,28) are newly discovered topologies from SeGen.

3.4 Advancements and Opportunities in Sequencing Element Synthesis

a) Study for further topology generation. The process of synthesizing sequencing elements offers valuable insights. The Boolean functions generated by SeGen cover all types of sequencing mechanisms and corresponding elements. Additionally, each Boolean function can produce various gate-level representations, which can be converted into distinct topologies with different design specifications (e.g., 18TSPC [2], S2CFF [11], REFF [17]). It implies that as long as the operation of an FF can be described as regular (static) Boolean logic, SeGen can find its topology or at least a similar implementation.

Currently, SeGen's compound gate implementation is solely based on CMOS logic. This helps guarantee static and reliable operation of generated FFs, but also limits the range of exploration. More diverse topologies can be discovered with the same Boolean function, by using gate implementation methods other than CMOS logic, such as dynamic logic (CSFF [12]), transmission-gates (STPL [7]) and pass-transistor logic (ACFF [20]).

b) Transistor sizing. Simulation results clearly show the flip-flop circuit's performance, such as power, speed, and reliability, is predominantly determined by the circuit topology. For example, the master–slave structure poses challenges in achieving the speed of a pulsed latch. In this study, the transistor sizes are fixed to investigate and compare various topologies without bias (similar to comparisons in prior research [10, 16, 17]). However, once the proper topology is determined, sizes of transistors can be fine-tuned for specific design targets based on simulation results, and this can be incorporated in the design automation process of a FF.

c) Other types of sequencing elements. SeGen can generate other types of sequencing elements, such as dual-edge FF, by adjusting the functionality qualification step of Boolean functions. The resulting topologies of such new types can be explored in a follow-up study.

4 CONCLUSION

We introduce SeGen—a tool that automatically identifies all types of sequencing elements. As the behavior of any digital system can be represented as Boolean functions, SeGen searches all truth tables qualifying the desired behavior, and the results include conventional master–slave FFs and pulsed latches as well as FFs with new mechanisms. SPICE simulation results on generated topologies provide a range of choices suitable for various applications. The proposed synthesis process outlined in this paper can be applied to any behavioral sequencing elements, such as dual-edge FF, with the adjustment of its state and qualification settings. This opens avenues for enhancing the efficiency and versatility of sequencing elements in digital integrated circuits.

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to Mr. Changjoo Park, Mr. Hyungjoon Bae, and Mr. Minkyu Yang from School of Electrical Engineering, KAIST for their invaluable contributions and support. This work was partly supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.2020-0-01297, Development of Ultra-Low Power Deep Learning Processor Technology using Advanced Data Reuse for Edge Applications, 50%) and Samsung Electronics Co., Ltd(Contract ID: MEM230315_0004, 50%). The EDA Tool was supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

[1] Massimo Alioto, Elio Consoli, and Gaetano Palumbo. 2015. Variations in Nanometer CMOS Flip-Flops: Part I—Impact of Process Variations on Timing. *IEEE Transactions on Circuits and Systems I: Regular Papers* 62, 8 (2015), 2035–2043.

[2] Yunpeng Cai, Anand Savanth, Pranay Prabhat, James Myers, Alex S. Weddell, and Tom J. Kazmierski. 2019. Ultra-Low Power 18-Transistor Fully Static Contention-Free Single-Phase Clocked Flip-Flop in 65-nm CMOS. *IEEE Journal of Solid-State Circuits* 54, 2 (2019), 550–559.

[3] D.J. Frank, R.H. Dennard, E. Nowak, P.M. Solomon, Y. Taur, and Hon-Sum Philip Wong. 2001. Device scaling limits of Si MOSFETs and their application dependencies. *Proc. IEEE* 89, 3 (2001), 259–288.

[4] Martin Charles Golumbic, Aviad Mintz, and Udi Rotics. 2008. An improvement on the complexity of factoring read-once Boolean functions. *Discrete Applied Mathematics* 156, 10 (2008), 1633–1636.

[5] Jiwoo Hong, Sunghoon Kim, and Dongsuk Jeon. 2022. An Automatic Circuit Design Framework for Level Shifter Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41 (2022), 5169–5181.

[6] Shailendra Jain, Surhud Khare, Satish Yada, V Ambili, Praveen Salihundam, Shiva Ramani, Sriram Muthukumar, M Srinivasan, Arun Kumar, Shasi Kumar Gb, Rajaraman Ramanarayanan, Vasantha Erraguntla, Jason Howard, Sriram Vangal, Saurabh Dighe, Greg Ruhl, Paolo Aseron, Howard Wilson, Nitin Borkar, Vivek De, and Shekhar Borkar. 2012. A 280mV-to-1.2V wide-operating-range IA-32 processor in 32nm CMOS. In *2012 IEEE International Solid-State Circuits Conference*. 66–68.

[7] Hanwool Jeong, Juhyun Park, Seung Chul Song, and Seong-Ook Jung. 2019. Self-Timed Pulsed Latch for Low-Voltage Operation With Reduced Hold Time. *IEEE Journal of Solid-State Circuits* 54, 8 (2019), 2304–2315.

[8] Dimitri Kagaris. 2016. MOTO-X: A Multiple-Output Transistor-Level Synthesis CAD Tool. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 35, 1 (2016), 114–127.

[9] Himanshu Kaul, Mark Anders, Steven Hsu, Amit Agarwal, Ram Krishnamurthy, and Shekhar Borkar. 2012. Near-threshold voltage (NTV) design — Opportunities and challenges. In *DAC Design Automation Conference 2012*. 1149–1154.

[10] Natsumi Kawai, Shinichi Takayama, Junya Masumi, Naoto Kikuchi, Yasuo Itoh, Kyosuke Ogawa, Akimitsu Ugawa, Hiroaki Suzuki, and Yasunori Tanaka. 2014. A fully static topologically-compressed 21-transistor flip-flop with 75% power saving. *IEEE Journal of Solid-State Circuits* 49, 11 (2014), 2526–2533.

[11] Yejoong Kim, Wanyeong Jung, Inhee Lee, Qing Dong, Michael Henry, Dennis Sylvester, and David Blaauw. 2014. 27.8 A static contention-free single-phase-clocked 24T flip-flop in 45nm for low-power applications. In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 466–467.

[12] Juhui Li, Alan Chang, Tony Tae-Hyoung Kim, et al. 2018. A 0.4-V, 0.138-fJ/cycle single-phase-clocking redundant-transition-free 24T flip-flop with change-sensing scheme in 40-nm CMOS. *IEEE Journal of Solid-State Circuits* 53, 10 (2018), 2806–2817.

[13] Markus Meissner and Lars Hedrich. 2014. FEATS: Framework for explorative analog topology synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 34, 2 (2014), 213–226.

[14] Vinicius Neves Possani, Vinicius Callegaro, André I. Reis, Renato P. Ribas, Felipe de Souza Marques, and Leomar Soares da Rosa. 2016. Graph-Based Transistor Network Generation Method for Supergate Design. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 24, 2 (2016), 692–705.

[15] Gicheol Shin, Minhyeok Jeong, Donguk Seo, Shin Han, and Yoonmyung Lee. 2022. A Variation-Tolerant Differential Contention-Free Pulsed Latch with Wide Voltage Scalability. In *2022 IEEE Asian Solid-State Circuits Conference (A-SSCC)*. 1–3.

[16] Gicheol Shin, Eunyoung Lee, Jongmin Lee, Yongmin Lee, and Yoonmyung Lee. 2020. A Static Contention-Free Differential Flip-Flop in 28nm for Low-Voltage, Low-Power Applications. In *2020 IEEE Custom Integrated Circuits Conference (CICC)*. 1–4.

[17] Gicheol Shin, Eunyoung Lee, Jongmin Lee, Yongmin Lee, and Yoonmyung Lee. 2021. An Ultra-Low-Power Fully-Static Contention-Free Flip-Flop With Complete Redundant Clock Transition and Transistor Elimination. *IEEE Journal of Solid-State Circuits* 56, 10 (2021), 3039–3048.

[18] V. Stojanovic and V.G. Oklobdzija. 1999. Comparative analysis of master-slave latches and flip-flops for high-performance and low-power systems. *IEEE Journal of Solid-State Circuits* 34, 4 (1999), 536–548.

[19] Y. Taur. 2002. CMOS design near the limit of scaling. *IBM Journal of Research and Development* 46, 2.3 (2002), 213–222.

[20] Chen Kong Teh, Tetsuya Fujita, Hiroyuki Hara, and Mototsugu Hamada. 2011. A 77% energy-saving 22-transistor single-phase-clocking D-flip-flop with adaptive-coupling configuration in 40nm CMOS. In *2011 IEEE International Solid-State Circuits Conference*. 338–340.

[21] A. Wang and A. Chandrakasan. 2004. A 180mV FFT processor using subthreshold circuit techniques. In *2004 IEEE International Solid-State Circuits Conference (IEEE Cat. No.04CH37519)*. 292–529 Vol.1.

[22] Jiren Yuan and C. Svensson. 1997. New single-clock CMOS latches and flipflops with improved speed and power savings. *IEEE Journal of Solid-State Circuits* 32, 1 (1997), 62–69.